RL78 串列 ISP：共用工程與協定契約

MCU 串列 ISP 工程配置

F13、F24、G16、G22 分別提供 UART、Xmodem、I2C 的獨立 Boot / App / Transfer。 每份 Device Boot 只編入所選的一種更新介面 / 協定；UART0保留診斷文字。 Tool操作與接線見串列 device 操作，本節集中於工程編輯與資料契約。 Tool選取值、Boot / App初始化、live INFO與BIN需對應的項目集中於通訊設定與更新必要條件。

配置與資料來源

F13 / F24 I2C App 與 Transfer 的 `custom_func.c` 擁有 UART0 console：

- `isp_console.inc` 在主迴圈輸出有效封包的 sequence、command、length 與完整 payload；ISR 不解析文字或執行 reset。
- F13 / F24 保存有上限的封包快照，診斷輸出在主迴圈處理。
- UART0 `z` / `Z` 只要求一般重置；資料介面完成目前回覆後才允許重置，I2C 包含最後 NACK / STOP。診斷 TX 停滯不會無限阻擋已可執行的重置。
- I2C backend 區分完整回覆與已放棄交易。最後活動後 2 秒仍未完成的 mailbox 會清除；App / Transfer 同步取消該回覆的待執行動作及未完成文字，不將放棄視為成功 ACK。
- 交易放棄後，明確輸入 UART0 `z/Z` 仍須通過 bus idle 與非更新中條件；不會自動重置，也不會沿用已取消的 ENTER 動作。
- Boot / App metadata、IRQ、Flash geometry、UART1 binary 與 I2C mailbox 格式維持各自契約。UART0 接線與操作集中於串列 device 操作。

MCU	CPU / 時鐘	Code Flash / RAM	App最大完整BIN	尾端保護區
F13	32MHz / X1 4MHz	128KiB / 8KiB	119KiB	0x1FC00..0x1FFFF
F24	40MHz / X1 4MHz	256KiB / 24KiB	247KiB	0x3FC00..0x3FFFF
G16	16MHz / HOCO	32KiB / 2KiB	23KiB	0x07C00..0x07FFF
G22	32MHz / HOCO	64KiB / 4KiB	55KiB	0x0FC00..0x0FFFF

各工程Boot為0x0000..0x1FFF（8KiB），建置檢查至少512-byte餘裕。 App表為0x2000..0x20FF，metadata為0x2100..0x21FF，程式從0x2200開始；完整BIN含上述內容、padding及末端4-byte CRC32。 尾端保護與安裝方式見 Flash尾端與debug monitor。

MCU	更新UART RX向量	IICA0向量	1-ms timer向量	Flash實作
F13	0x40	0x28	0x1E	FSL Type01，Flash busy鏈位於RAM
F24	0x40	0x28	0x2E	RFD Type02，Flash busy鏈位於RAM
G16	0x20	0x2C	0x1C	RFSP Type01，Code Flash操作期間CPU停止執行

MCU	更新UART RX向量	IICA0向量	1-ms timer向量	Flash實作
G22	0x26	0x2A	0x2E	RFD Type01，startup複製Flash driver到RAM

向量值是MCU實體vector的byte address，來源為對應硬體手冊的向量表及工程iodefine.h。 G16 / G22 UART1使用SAU0 channels2 / 3；實際pin與PIOR值由各board adapter管理。Flash函式的ROM儲存與RAM執行概念見 Flash執行位置。

UART速率由 `isp_transport_config.h` 宣告；INFO、metadata及周邊分頻需一致。 `isp_bus_contract.h` 檢查各宣告值；SAU adapter依目前選定的UART source clock產生SDR，`isp_uart_timing.h` 拒絕無法表示或誤差超過2%的組合。UART0 Log維持獨立分頻。 若速率需要不同的SPS時脈來源，先調整board adapter與其source clock，再重建配對Boot / App；設定檢查通過不代替實際電氣與通訊時序驗證。

編輯、產生與建置

輸入 / 來源	用途與編輯時機
App app_image_config.h	發行version、build、target、DEVICE_APP、update transport、version control；每個define附註解
App app_irq_handlers.h	實體vector與global ISR mapping；新增中斷時編輯
App custom_func.c	hardware_init / loop 入口、業務功能、APP_LOG_TAG與counter觀察；入口與產碼說明
各工程 image_metadata_config.h	MCU / Flash、App起點 / 容量、格式、IRQ ABI、Boot identity
各工程 isp_transport_config.h	UART / I2C / Xmodem的編譯契約、速率、位址、timeout
各工程 boot_app_handoff.h	owner與RAM保留區、ENTER / 更新完成marker
工程同名 .scfg	SC適用工程的MCU、時脈、UART / IICA與腳位配置，見SC設定與重新產生
image_metadata.c/.h、image_metadata_profile.c	中性parser與expected profile；不操作UART / I2C / Flash
產生的INC / identity / vectors	Pre-build輸出；由上述C headers產生，不另行手改

Boot / App / Transfer 各自包含 API、設定 header 與建置工具，不依賴相鄰工程。 目錄分工見 工程目錄與程式入口。 在所選工程根目錄執行：

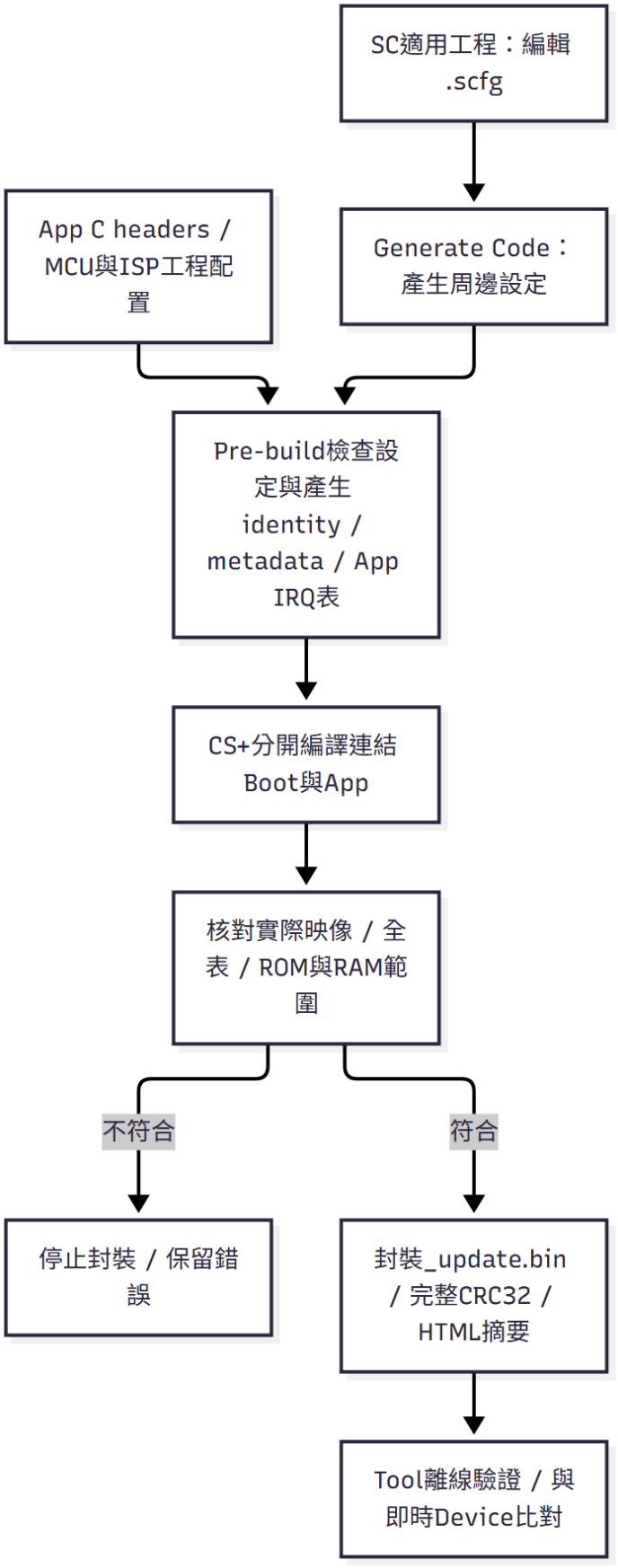
```
powershell -NoProfile -ExecutionPolicy Bypass -File .\build.ps1
```

使用CS+ / CC-RL與Windows PowerShell，不增加Python或JSON authoring前提。 `build.ps1` 只建置所在工程；`-Compiler` 可指定既有 CubeSuite+.exe 位置。

Pre-build產生完整表與metadata；post-build核對實際linked image、ISR返回方式、CRC、ROM / RAM範圍後產生_update.bin與app_build_report.html。 HTML列出實際BIN的metadata bytes、IRQ、CRC與雜湊；它是編譯報告，不是板上更新結果。

- F13 I2C App 以 `.boot_info` 保留 `0xFF710..0xFF71F` 的 no-init 交接 RAM；owner、HANDOFF、更新完成與返回 Boot 標記不由 startup 清除。
- F13 App post-build 以實際 map 核對交接區、堆疊及一般 RAM 初始化範圍互不重疊，且堆疊至少保留 512 bytes；動態堆疊用量仍需依 App 工作負載驗證。各標記的生命週期與位址差異見 RAM 交接契約。

編輯、產生與建置


SC適用工程：編輯 .scfg
App C headers / MCU與ISP工程配置
Generate Code：產生周邊設定
Pre-build檢查設定與產生 identity / metadata / App IRQ表
CS+分開編譯連結Boot與App
核對實際映像 / 全表 / ROM與RAM範圍
不符合
符合
停止封裝 / 保留錯誤
封裝_update.bin / 完整CRC32 / HTML摘要
Tool離線驗證 / 與即時Device比對

Boot持有完整實體vector；AA選Boot，55經App表轉送。 App ISR為global、無vect=的interrupt pragma，並列於app_irq_handlers.h。 同一專案不能重複定義同名ISR；分開連結的Boot / App可以同名。 新增IRQ亦包含C來源註冊、周邊初始化與旗標清除；完整流程見 App IRQ編輯。

Metadata offset與協定配置

總長256bytes，core128bytes、其餘128為FF。除ASCII與byte欄位外，多byte整數採little-endian。 下列offset相對於metadata起點0x2100；例如version在0x2110，policy在0x2174。

Offset	Bytes	內容 / 來源
00	4	49 53 50 00，ISP加NUL；不是周邊名稱
04	4	0x00038006：format6、core128、flags3
08	4	0x01010055 UART、0x01010049 I2C、0x01010058 Xmodem
0C	4	MCU target：F13B / F24B / 016B / 022B
10 / 14	各4	App version / build，來自App header
18	4	完整BIN實際長度，由post-build產生
1C / 20 / 24 / 28 / 2C	各4	App起點 / 容量 / Code Flash / Boot / erase bytes
30 / 34 / 38 / 3C	各4	I2C為編譯Hz / 7-bit address / 0 / 0，範例100000 / 20h / 0 / 0；UART與Xmodem皆0
40..4F / 50..5B	16 / 12	MCU part / family，ASCII與NUL padding
5C	4	最小完整BIN516bytes
60	4	UPD1 policy magic
64	4	UART / Xmodem=04012003h，I2C=02012003h；policy3、32bytes、DEVICE_APP與允許transport
68 / 6C	各4	MCU IRQ ABI / 最低Boot版本
70	4	UART / Xmodem編譯baud，範例115200；I2C=0
74	4	version control：0任意版本、1同版 / 新版
78..7F / 80..FF	8 / 128	零 / FF

版本號採00MMmmpp，build分開回報；最低Boot不是App / Boot版號相等要求。 只有完整驗證通過的installed App提供版本下限；FORCE_START只略過該交易版本排序，其他格式、identity、layout、IRQ與CRC檢查不變。

SC設定與重新產生

F24、G16、G22 的 UART / Xmodem / I2C Boot、App、Transfer各自帶有同名 `.scfg`。 F13使用工程內的Code Generator與board設定，沿用相同的metadata與App IRQ編輯方式。

配置	SC元件	Runtime負責
UART / Xmodem	UART0 Log、UART1更新、TAU0_1一毫秒tick、r_bsp時脈資料	有界RX queue、協定處理與傳送完成判斷
I2C	UART0 Log、IICA0 Slave、TAU0_1一毫秒tick、r_bsp時脈資料	Mailbox、STOP / NACK與回覆完成後的reset條件

Xmodem與UART使用相同周邊配置；XMODEM-CRC128的資料處理由協定程式負責。 各MCU的腳位見工程README；G16的16MHz時脈依據原廠器件規格。

1. 在CS+開啟所選工程，進入Smart Configurator節點，開啟工程同名 `.scfg`。
2. 修改通訊設定時，同步核對配對Boot的 `isp_transport_config.h`，以及Boot / App / Transfer各自的SC設定。 UART1 baud與編譯契約一致；UART0保留獨立Log設定。
3. I2C契約與Tool使用7-bit位址。SC的address欄位對應SVA暫存器值：`SVA = 7-bit address << 1`。 例如7-bit `0x20` 對應SVA `0x40`，SC十進位輸入值為 `64`；7-bit `0x42` 對應SVA `0x84`，十進位值為 `132`。
4. 在SC執行 **Generate Code**，再於CS+執行 **Rebuild**，或使用上方PowerShell建置命令。
5. 核對建置輸出的 `PASS SC MCU/pins/clocks/serial/timer and actual generated registers`。 App建置完成後，再開啟 `DefaultBuild/app_build_report.html` 檢查實際BIN的metadata與IRQ內容。

Pre-build會對照 `.scfg`、實際 `src/smc_gen` 產物與C header契約， 將核對過的時脈、UART0分頻、timer及IICA timing輸出至工程自己的 `sc_peripheral_config.generated.h`，供runtime初始化使用。 此header是產物，不另行編輯；修改設定後，透過SC重新產生及Rebuild更新。

若出現 `SC consistency` 或 `Regenerate SC`，建置會在產表與封裝前停止。 訊息會指出MCU、腳位、時脈、速率、位址或尚未更新的產物；調整對應輸入後再產生。 目前board adapter固定管理所列的clock與pin路由；若改用其他腳位或clock來源，也需修改adapter及其檢查條件。

工程保留自己的startup與IRQ入口。SC的周邊設定透過上述header接入； UART / IICA預設callback ISR不另外加入連結，App也不產生第二套實體vector。 App新增中斷仍由App IRQ編輯流程處理。 SC重新產生與建置檢查不代表已完成板上通訊、動態stack或Flash操作驗證。

Framed control與INFO

Frame byte	Bytes	意義
0..1	2	49 53 marker
2 / 3	各1	framing version1 / request0或response1
4..5	2	sequence LE16，1..65535
6 / 7	各1	command / status，request status0
8..9	2	payload長度LE16，0..96
10..	0..96	payload
payload之後	4	header與payload的CRC32，LE32

CRC32/ISO-HDLC：reflected polynomial EDB88320，initial / final XOR FFFFFFFF；123456789=CBF43926。 Frame最長110bytes；UART partial frame200ms逾時。相同sequence僅允許完全相同request重播。

Command	作用
01 INFO	88-byte live contract / mode / MCU / geometry / installed App / capabilities
02 ENTER	回覆完成後reset到Boot；Boot直接ACK
03 METADATA	LE16 offset加4..64bytes，依序stage256，不擦Flash
04 START / 0E FORCE_START	LE32完整size加LE32末端CRC；先驗證metadata與policy
05 WRITE / 06 END	UART / I2C：ordered64-byte write與完整映像驗證；Xmodem Boot拒絕，改由raw block / EOT處理
07 ABORT	清session / staging，不復原已擦寫內容
08 READ_METADATA / 09 VERIFY	有效App metadata讀回 / Boot唯讀重算完整映像
20 PING / 21 ECHO	App / Transfer回傳0..96bytes
22 STATS / 23 CLEAR_STATS	App / Transfer四個LE32：requests、replays、errors、written
24 STREAM_TEST	Xmodem App / Transfer宣告size1..65536與CRC32，轉到RAM診斷sink
25 DIAGNOSTICS	宣告能力的UART / Xmodem App / Transfer，回覆唯讀24-byte UART快照

以下 offset 以 INFO payload byte 0 為基準，多 byte 整數使用 little-endian。

INFO payload：88 bytes

Offset	Bytes	欄位與意義
00	1	contract：UART = 1、I2C = 2、Xmodem = 3
01	1	mode：Boot = 0、App = 1、Transfer = 2
02	1	transport：I2C = 02、UART / Xmodem = 04
03	1	max-write：UART / I2C = 64、Xmodem = 128
04..05	2	MCU target ID
06 / 07	各1	installed App valid / 更新 active
08..0B	4	App 起點
0C..0F	4	App 完整 BIN 容量
10..13	4	Code Flash bytes
14..17	4	Boot 保留 bytes
18..1B	4	Flash erase bytes
1C..1F / 20..23	各4	Boot version / build
24..27	4	編譯 bus rate：UART baud 或 I2C Hz
28..2B / 2C..2F / 30..33	各4	installed App version / build / 完整 BIN size
34 / 35	各1	installed version-control policy / App ready
36..37	2	capabilities bitmap
38..47	16	part ASCII，NUL padding
48..53	12	family ASCII，NUL padding
54..57	4	IRQ ABI

Boot能力0F、App3C、Transfer10；Transfer不回報installed App / Boot分區與IRQ ABI，其對應值為零。 提供UART診斷callback的App / Transfer另加能力bit6（0x40）；Boot不加此位元。

更新命令 payload

Command	Payload offset	Bytes	意義
03 METADATA	00..01	2	Metadata offset，4-byte 對齊
03 METADATA	02..	4..64	Metadata bytes；每段為4的倍數，依序涵蓋256 bytes
04 START / 0E FORCE_START	00..03	4	完整 BIN bytes，包含向量、Metadata、padding 與末端 CRC
04 START / 0E FORCE_START	04..07	4	BIN 末端保存的 CRC32 值
05 WRITE	00..03	4	從 BIN 起點計算的 offset
05 WRITE	04..	4..64	UART / I2C 本次寫入 bytes；非末塊64 bytes，末塊使用實際剩餘長度
06 END / 07 ABORT / 09 VERIFY	無	0	END 完整驗證、ABORT 清交易、VERIFY 唯讀驗證
08 READ_METADATA	00..01 / 02	2 / 1	Metadata offset / 要求 bytes；須為有效 installed App
24 STREAM_TEST	00..03 / 04..07	各4	Xmodem App / Transfer 的 useful size / CRC32

成功回覆保留 request 的 sequence 與 command，type = response、status = 00；有資料時依命令附 payload。 可解碼但被拒絕的命令回覆錯誤 status 與空 payload。

Status 與錯誤處理

Status	名稱	作用與核對方向
00	OK	此命令已接受；更新完成仍需 END / EOT 驗證及 App 確認
01	BAD_COMMAND	命令不屬於目前角色、協定或能力
02	BAD_SIZE	payload、offset 對齊或資料區塊長度不符合條件
03	BAD_STATE	尚未完成 Metadata staging、未 START 或目前狀態不接受命令
04	BAD_SEQUENCE	命令 sequence / 重播內容或 Metadata / BIN offset 不符合順序
05	BAD_METADATA	MCU、介面、配置、size、IRQ 或映像契約不符合
06	BAD_VERSION	有效 installed App 啟用版控，且 incoming version 較低
07	FLASH_ERROR	實際擦除或寫入 API 失敗
08	CRC_ERROR	完整映像或 VERIFY 檢查未通過

無法形成有效 frame 的資料不執行命令，也不保證有 status 回覆。 Status 名稱與值來自工程內 `hal/isp/isp_protocol.h`，命令條件由 `isp_device.c` 核對。

各介面 Bootloader 與協定格式

介面	Boot 流程	Protocol Format
UART	接收、中斷與更新	UART frame 與診斷
I2C	Slave mailbox 與更新	I2C request / reply window
Xmodem	Control / raw block 與完成	XMODEM-CRC128

驗證範圍

共用驅動修正須分別檢查 MCU、介面與 Boot / App / Transfer 的適用範圍。 來源同步、目標建置、更新 BIN 與容量測試 BIN 是不同產物；同步來源後，既有 BIN 仍保有原本的執行內容。

- F13 / F24 I2C 工程各自編譯本地 driver；Boot 與 App 都需納入修正評估。
- App-only 建置保留配對 Boot；Boot 更新另依安裝流程處理。
- App 重建後重新產生容量測試 BIN，核對來源雜湊、CRC、長度與保護區；Tool 重新載入該檔案。
- 其他介面及 RH850 依實際 driver / IRQ / timer 判斷，不直接套用 RL78 暫存器對策。

操作與容量測試順序見Tool I2C 流程。

F24 I2C 一般更新已涵蓋原始 App 與 8 / 24 / 56 / 120 / 247 KiB 映像： END 成功、App ready 與全部 installed metadata 相符，UART0 也有重啟後持續運作紀錄。 環境、分階段耗時及限制集中於F24 I2C 更新量測。 此結果不延伸為其他 MCU / 介面、冷啟動、斷電恢復或原始異常根因的驗證。

離線檢查包含實際App BIN、production PC client、C engine、transport loop / receiver、四系列IICA register模型，以及CS+ linked metadata / vector / ROM / RAM guards。App-only建置會比對Boot來源與產物雜湊，確認未被改寫。 這些檢查不代替實板電氣、Flash timing、中斷、動態stack、重啟與斷電測試。

UART Bootloader 與協定格式

本章使用 framed UART ISP；每份 Boot、App、Transfer 各自是獨立工程。 UART 資料與診斷 Log 分開接線；Tool 的 Host Baud 與 Device baud 也分別設定。

工程與接線

用途	工程或連線	詳細說明
Boot / App / Transfer	選擇名稱含 UART 的對應工程；各自包含來源與建置工具	平台工程與周邊
ISP 資料	Bridge 資料 UART TX 接 device RX，RX 接 TX；共地並核對電壓	Tool 接線與操作
診斷文字	Device 的獨立 Log UART 接 terminal	平台工程與周邊
建置與下載檔	在所選工程執行 PowerShell 建置；Boot 安裝檔與 App 更新 BIN 分別使用	編輯、產生與建置

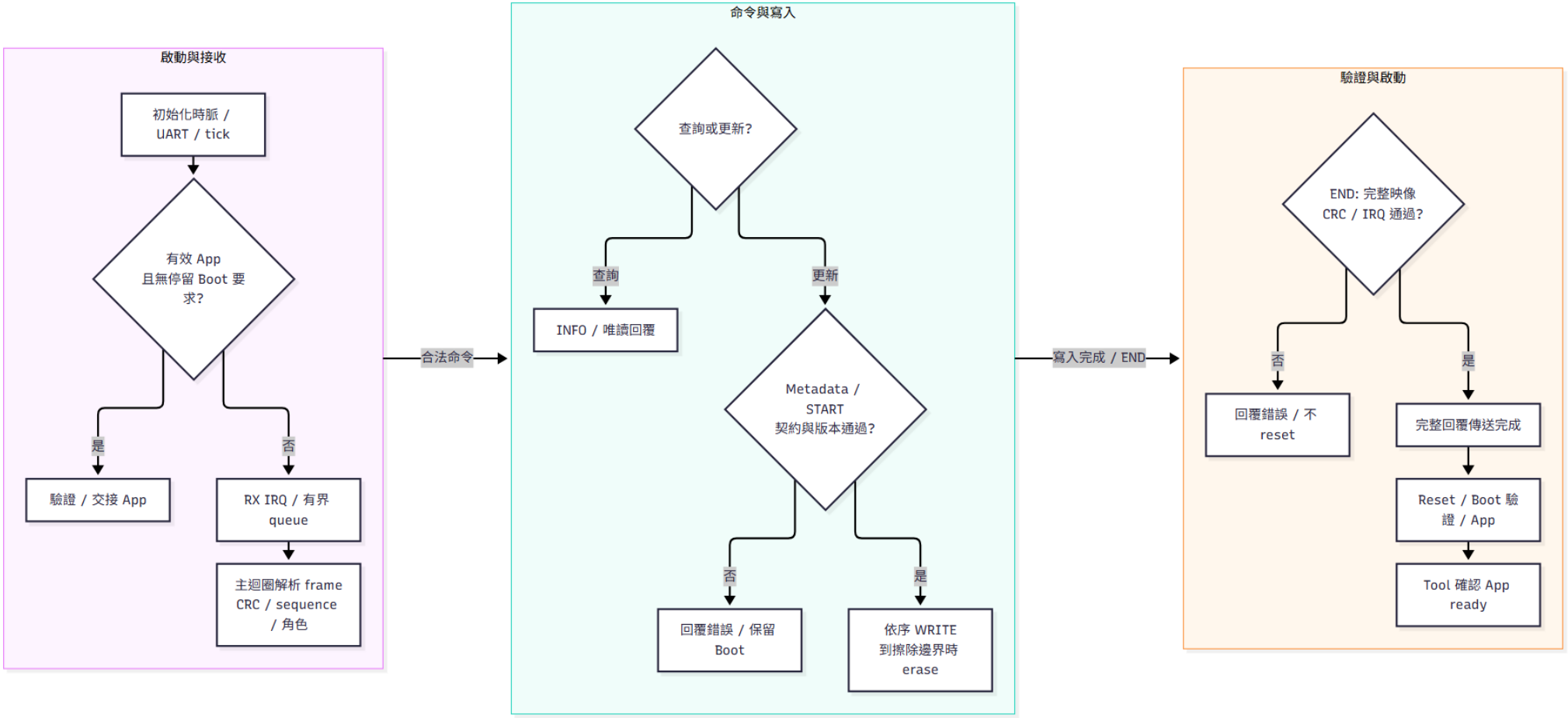
App-only 建置使用 App 自己的契約與工具；安裝到板上的 Boot 仍需與 App 相容。 Custom Transfer 是獨立通訊診斷映像，不擦寫 App Flash。

通訊設定

修改來源	作用	同步核對
`isp_transport_config.h` 的 `ISP_UART_BAUD`	Device 資料 UART 速率，範例為 115200 bit/s、8-N-1	配對 Boot / App 的周邊分頻、INFO、BIN Metadata 與 Tool Device baud
`ISP_POLICY_UART_BAUD`	映像契約宣告的 UART baud	與實際初始化及上述速率相符
`.scfg` 或各工程 board adapter	周邊、腳位、來源時脈與分頻	SC 設定與重新產生
`ISP_INTERBYTE_TIMEOUT_MS`	未完成封包的接收閒置期限，範例 200 ms	與實際傳輸及接收處理相符
`ISP_BOOT_IDLE_TIMEOUT_MS`	有效 App 進入 Boot 後、更新尚未開始時的閒置返回期限，範例 30000 ms	已開始的更新不套用此閒置返回

變更 baud 時重建配對 Boot / App，並安裝更新後的 Boot。 可選速率與必要條件集中於 通訊設定與更新必要條件。 Metadata 欄位位置見 Metadata offset 與協定配置。

Boot 執行流程



INFO 等唯讀命令也由主迴圈回覆；ISR 只處理接收資料與錯誤狀態。 Metadata staging 與 START 不擦除 Flash；第一個合法 WRITE 才依擦除邊界啟動 Flash 操作。

Protocol Format

UART 依 byte stream 傳送共用 ISP frame。接收端依 header 的 payload length 找到 CRC32 尾碼， 不以 UART idle、換行或 terminal 文字作為命令結尾。

層級	內容與長度	參考
Frame	10-byte header + 0..96-byte payload + 4-byte CRC32，最長 110 bytes	逐 byte 封包與命令表

層級	內容與長度	參考
METADATA `03`	LE16 Metadata offset + 4..64 bytes；依序暫存全部 256 bytes	Offset 基準是 Metadata 起點
START `04`	LE32 完整 BIN bytes + LE32 映像末端 CRC32	驗證已暫存 Metadata、配置與版本政策
WRITE `05`	LE32 BIN offset + 最多 64 bytes；非末塊為 64 bytes	Offset 基準是 BIN 起點，實體位址 = App 起點 + offset
END `06`	空 payload	重新驗證完整已寫入映像，再回覆結果

CRC32 有兩個用途：frame 尾碼保護本次 header / payload；BIN 末端 4 bytes 保護映像中尾碼之前的內容。 兩者分別計算，不能以封包 CRC 正確推定 Flash 映像有效。

完整封包範例

以下 INFO request 為 sequence = 1、command = 01、空 payload；可以送給 Boot、App 或 Transfer。 最後四個 bytes 是前十個 bytes 的 CRC32，採 little-endian。

```
49 53 01 00 01 00 01 00 00 00 F1 8D B0 75
```

具體回覆的 88-byte INFO payload 取決於板上角色、MCU、分區與 installed App；欄位定義見共用契約。

成功與錯誤回覆範例

下列為依格式計算的封包範例，不是板端擷取記錄。以Transfer角色為例，PING將payload原樣回覆；未知命令回覆`BAD_COMMAND=1`，兩者的sequence / command都對應request。

```
PING request: 49 53 01 00 02 00 20 00 03 00 12 34 56 1F 91 6C 3F
PING reply:   49 53 01 01 02 00 20 00 03 00 12 34 56 21 FA AE D0
Unknown request: 49 53 01 00 03 00 7F 00 00 00 55 FB 98 B3
Error reply:     49 53 01 01 03 00 7F 01 00 00 D6 9A 2D 14
```

Header的response byte由00改為01；status為00成功、01未知命令。

每列最後4bytes分別對該列header與payload計算CRC32；錯誤回覆也有自己的有效CRC。

異常與完成條件

觀察	Device / Tool 行為	核對方向
Frame 未收齊或 CRC 錯誤	不執行該命令；partial frame 超過設定期限後清除	baud、接線、queue / 硬體錯誤、封包長度
同一 sequence 重送	只接受完整相同 request，重播快取回覆	不把重播當作第二次寫入
START / WRITE 被拒絕	回覆對應 status，不以錯誤 request 授權 Flash 操作	狀態碼與命令條件
END 驗證成功	完整回覆傳送完成後才 reset	Device 傳送完成與 Tool 收到回覆分別判斷
App 確認尚未成功	保留報告並讀取即時狀態	不自動重複 erase / END / RUN 來推定結果

實際操作與報告判讀見 Tool 串列操作。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`	平台初始化、封包處理及回覆完成後的交接順序
`hal/isp/isp_protocol.c`	`IspStreamFeed()` / `IspEncode()`	串流組包、長度與 frame CRC32
`hal/isp/isp_device.c`	`IspDeviceProcess()` / `IspDeviceWrite()` / `IspDeviceFinish()`	命令、角色、版本、範圍及更新狀態
`hal/device`	各平台 UART RX / Error ISR、board API	有界 queue、周邊初始化與傳送完成
`hal/image` / `hal/flash`	映像驗證與 Flash API	Metadata、完整 CRC、IRQ 與實際擦寫

UART診斷格式

Command0x25不帶payload。只有INFO宣告bit6時才查詢；回覆是下表24bytes，多byte整數採little-endian。 回覆使用相同sequence快取規則；重播同一request不重新讀取硬體。

Offset	Bytes	內容與範例
00	1	格式1
01	1	診斷種類1：Device UART
02..03	2	欄位mask0x000F，代表本表四組計數 / queue欄位均提供
04..07	4	RX硬體錯誤次數
08..0B	4	RX queue丟棄新資料次數
0C..0F	4	本機TX timeout次數
10..13	4	RX queue最高使用bytes
14..17	4	快照當下RX queue待處理bytes

RL78 硬體錯誤 / 丟棄 / TX 計數飽和於 65535，欄位統一序列化為 32 bits。 最高量與待處理量是bytes，不是CAN frame數；reset後重新起算，不宣告跨重啟的累積量。 快照只短暫保留並還原對應IRQ遮罩，不清空接收queue或重設周邊。 `CLEAR_STATS`只清除協定計數，不清除此硬體診斷快照的累積量。 Tool保留支援 / 未提供 / 讀取失敗的區別；畫面與報告的用途見 Device串列診斷。

I2C Bootloader 與協定格式

Device 使用 I2C Slave，bridge 使用 I2C Master。每份 Boot、App、Transfer 各自是獨立工程， 另保留獨立 UART 診斷文字。

工程與接線

用途	工程或連線	詳細說明
Boot / App	選擇名稱含 ISP_Boot_loader_I2C 的配對工程	平台工程與周邊
Custom Transfer	名稱含 I2C_Slave_Transfer 的獨立診斷工程	透過相同 mailbox 驗證 PING / ECHO / 計數，不擦 Flash
I2C 資料	SCL、SDA、共地；核對上拉電壓、阻值與負載	Tool 接線與操作
診斷文字與產物	獨立 Log UART；Boot 安裝檔 / App 更新 BIN 分開使用	編輯、產生與建置

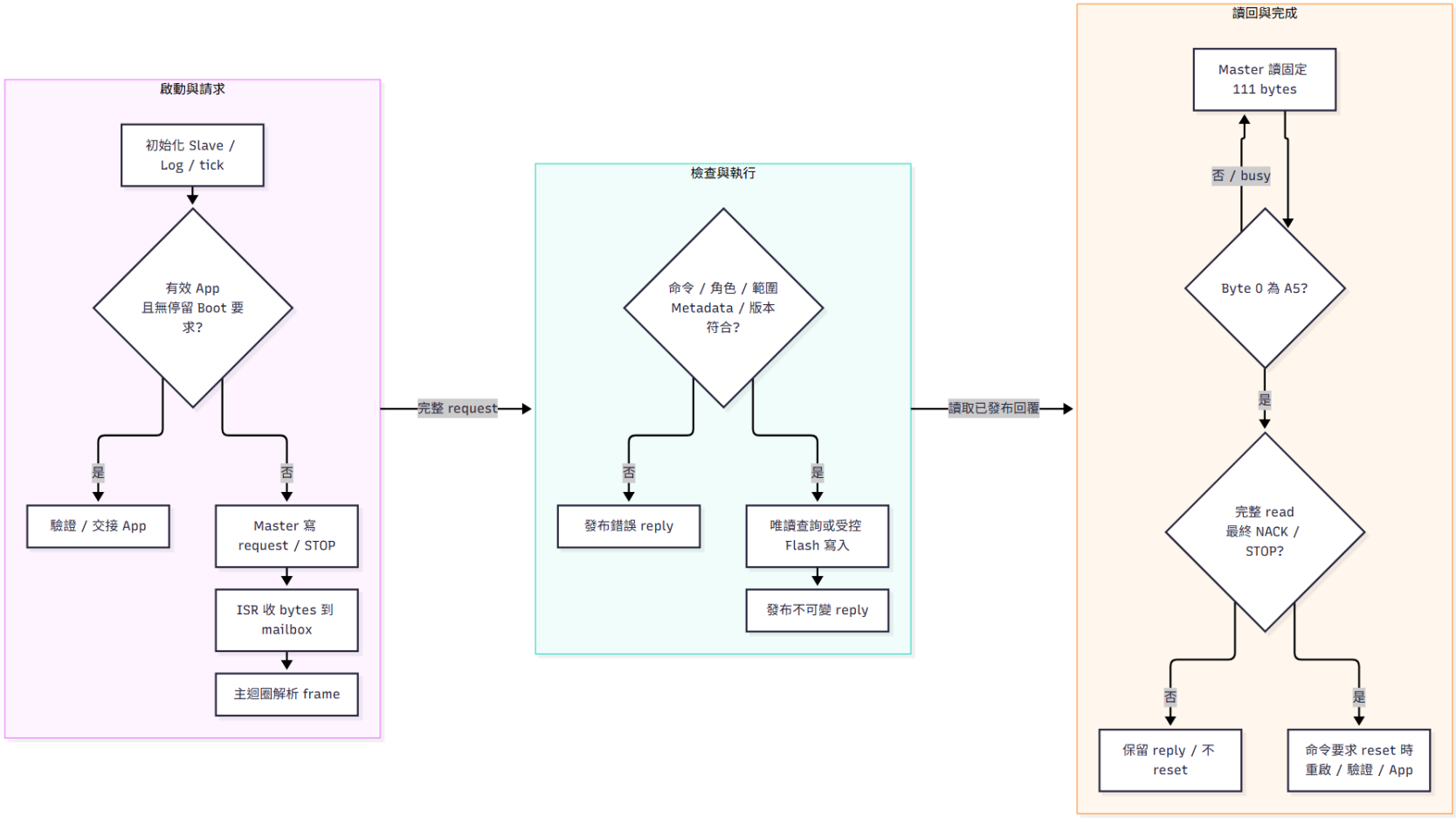
各工程包含自身建置來源，不需要其他工程 checkout 才能編譯。 燒錄後配對的 Boot / App 保持相同介面及映像契約。

通訊設定

修改來源	作用	同步核對
`ISP_I2C_ADDRESS` / `ISP_LINK_PARAMETER_1`	7-bit Slave address，範例 0x20	Boot / App slave 初始化、BIN Metadata + 0x34 與 Tool 所選位址
`ISP_BUS_RATE` / `ISP_LINK_PARAMETER_0`	編譯 bus rate，範例 100000 Hz	Slave 時序、INFO、BIN Metadata + 0x30 與 Tool 100 / 400 kHz 能力
`.scfg` 或各工程 board adapter	IICA、腳位、時脈與中斷	平台配置、SC 設定
`ISP_BOOT_IDLE_TIMEOUT_MS`	有效 App 進入 Boot 後、更新尚未開始時的閒置返回期限	範例 30000 ms；active 更新不套用此條件

Tool 輸入 7-bit 位址 0x20，不是 address byte 0x40。 完整設定條件集中於 通訊設定與更新必要條件。 INFO 不含 Slave address；Tool 使用實際交易位址，並核對讀回的 Metadata。

Boot 執行流程



START 檢查完整 staged Metadata；WRITE 依序寫入，END 驗證完整映像。 I2C ACK 只代表 bus byte / 交易的接收，更新結果仍以 framed reply status 與 App 確認判斷。

階段	完成條件	等待或失敗時的處理
ENTER	完整成功 reply 經最終 NACK / STOP 後才要求 reset	回覆未確認不推定已進 Boot；新 BIN 尚未傳送
WRITE	對應 sequence / command 的成功 reply	保留最後確認 offset；不把 bus ACK 當作 Flash 成功
END	完整映像驗證及成功 reply	保留 status 與驗證耗時；不重送 END

階段	完成條件	等待或失敗時的處理
重啟後	App ready 及全部 installed metadata 相符	與 END 成功分開判斷；依已安裝 / 新映像所在階段排程等待

大小估計與靜默查詢排程見Tool I2C 更新與等待。 Mailbox 的最後活動與逾時取樣須使用一致的 IRQ 保護，避免 ISR 更新活動時間後，主迴圈以較舊時間誤判逾時。 真正超過 2 秒未完成的交易仍會放棄；App / Transfer 取消該回覆的待執行動作與診斷所有權，不因此自動 reset。

Protocol Format

I2C 先寫入一份以 STOP 分隔的完整 request，再讀取固定 111-byte reply window。 ISR 只處理 bytes 與 START / NACK / STOP；主迴圈才解析、執行 Flash 並發布不可變回覆。

交易 / byte	長度與內容	意義
Write	START + address/W + 完整 ISP frame + STOP	Frame 為 10-byte header + 0..96 payload + CRC32，最長 110 bytes
Read byte 0	00 busy 或 A5 ready	只有 A5 後才解析 framed reply
Read byte 1..N	完整 framed reply，14..110 bytes	sequence / command 對應 request，status 回報命令結果
Read 剩餘 bytes	FF padding，直到總計 111 bytes	由內層 frame length 決定有效資料，不把 padding 納入 frame CRC
Read 結尾	Master 最終 NACK，接著 STOP	與完整 read 一起作為回覆完成條件

完整 header、各命令 payload、INFO 與 status 定義見 共用封包與命令表。 UART / I2C 使用相同命令格式，I2C 外層 mailbox byte 不屬於內層 CRC 範圍。

位址與交易範例

7-bit address = 0x20 時，線上 address byte 為寫入 0x40、讀取 0x41。 以下以 INFO request 為例；[ ] 表示說明佔位，不是線上 bytes。

```
S 40 49 53 01 00 01 00 01 00 00 00 F1 8D B0 75 P
S 41 00 [busy read window] NACK P
S 41 A5 [   INFO framed reply] [FF padding] NACK P
```

S 為 START，P 為 STOP；每次 read window 都讀取 111 bytes。 busy 後只重讀 mailbox，不重新送出前一個 Flash request。

成功與錯誤回覆範例

I2C內層沿用 UART完整request / reply範例。 以PING為例，ready window為A5 + 上述PING reply + FF padding至111bytes；未知命令則為A5 + error reply + padding。 A5只表示已有完整回覆；更新是否獲接受仍看內層status。 busy window的首byte為00，不是BAD_STATE或CRC錯誤回覆。

異常與完成條件

觀察	Device / Tool 行為	核對方向
無 address ACK	尚無有效 ISP 回覆	7-bit 位址、接線、電壓、上拉、Slave 初始化
Byte 0 = 00	唯讀 poll 等待 ready	不因 busy 重送 Flash request
Short read	保留 reply；不執行 pending reset	固定 111-byte read、最終 NACK 與 STOP
CRC / sequence / 命令錯誤	不以錯誤交易授權 Flash；可解碼命令回報對應 status	共用狀態碼
回覆已完整讀取	僅已接受命令的 pending action 可執行	ENTER / END 的重啟條件與 Tool App 確認

mailbox 的交易 lease 為 2 秒；逾時清除 mailbox 狀態不表示恢復已擦寫的 App。 更新 / 取消與報告見 Tool 串列操作。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`	初始化、主迴圈命令處理與 pending action
`hal/isp/isp_i2c_mailbox.c`	`IspMailboxStart()` / `IspMailboxReceive()` / `IspMailboxNext()` / `IspMailboxNack()` / `IspMailboxStop()`	有界交易資料與狀態，不解析 ISP 命令
同一 mailbox 模組	`IspMailboxGet()` / `Publish()` / `Release()`	主迴圈取 request、發布及釋放 reply
`hal/isp/isp_protocol.c` / `isp_device.c`	`IspStreamFeed()` / `IspDeviceProcess()`	內層 frame 與更新命令
`hal/device` 與各平台 I2C adapter	IICA ISR、reply-complete 判斷	暫存器、STOP / NACK / 最後資料的完成邊界

各平台所用 peripheral、實體向量 / EIINT 與 Flash API 集中於 平台配置。 Metadata、IRQ 與 Flash 驗證維持各平台工程的契約。

Xmodem Bootloader 與協定格式

本章使用 framed ISP control 搭配 XMODEM-CRC / 128-byte block。 Tool 在 **UART / Xmodem → Workflow** 選擇對應功能；Device 使用獨立 Xmodem 工程，另保留 Log UART。

工程與接線

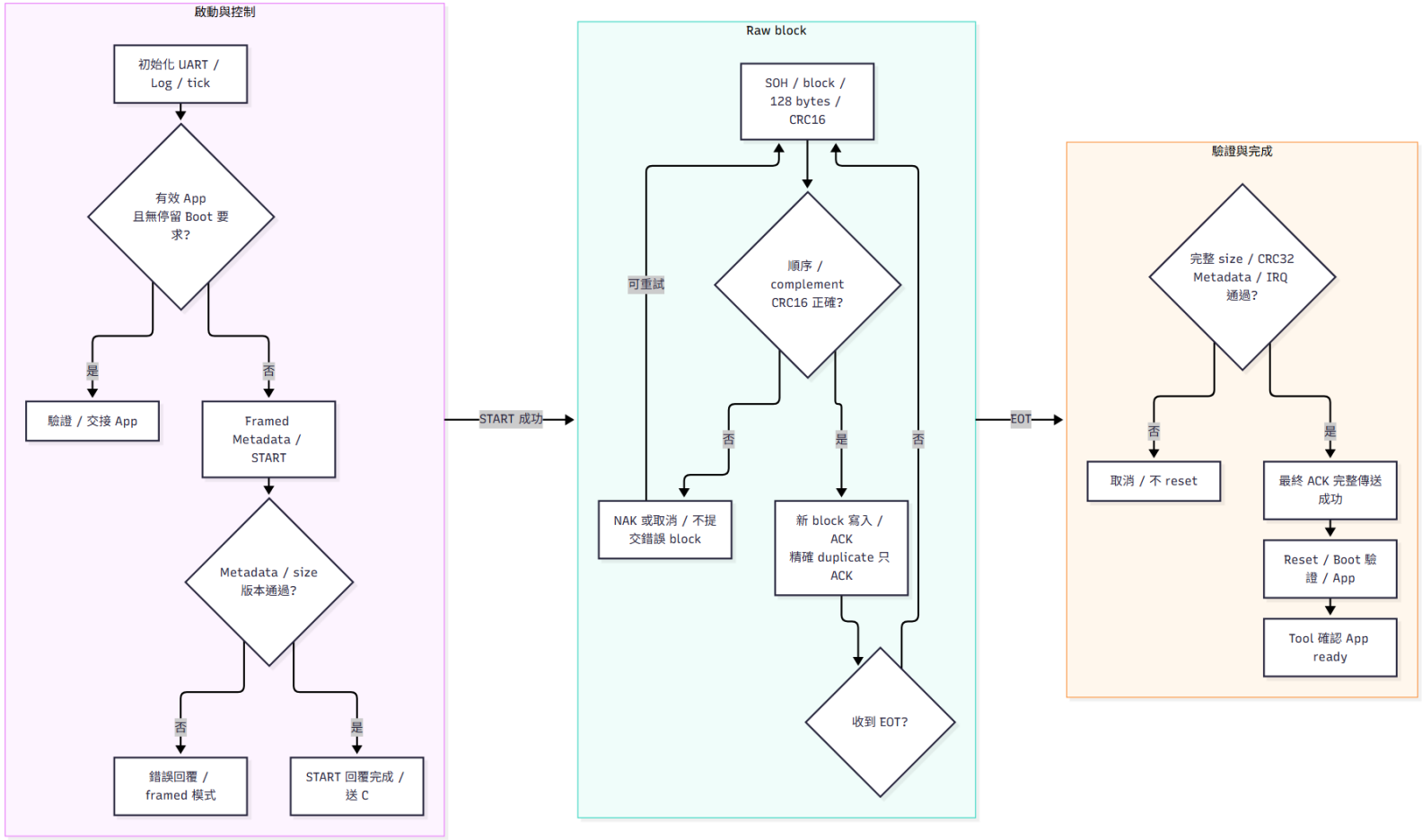
用途	工程或連線	詳細說明
Boot / App / Transfer	選擇名稱含 Xmodem 的對應工程，各自可獨立建置	平台工程與周邊
資料與文字	資料 UART 與獨立 Log UART 分別接線	Tool 接線與操作
App 更新	Tool 載入封裝的 `_update.bin`，以 XMODEM-CRC Workflow 更新	建置產物
Transfer 診斷	使用 STREAM_TEST 接收 RAM CRC32 sink	不擦 Flash；不回報已安裝 App 的成功更新

通訊設定

修改來源或條件	作用	同步核對
`ISP_UART_BAUD` / `ISP_POLICY_UART_BAUD`	資料 UART 速率，範例 115200 bit/s、8-N-1	配對 Boot / App 分頻、INFO、Metadata 與 Tool Device baud
`.scfg` 或 board adapter	UART、pin、clock 與接收中斷	SC 設定與重新產生
XMODEM-CRC128	固定 128-byte block，加 CRC16/XMODEM	與 Tool 的 XMODEM-CRC Workflow 相符
`ISP_BOOT_IDLE_TIMEOUT_MS`	更新尚未開始時、有效 App 的 Boot 閒置窗口	範例 30000 ms；raw streaming 另有接收期限

Metadata 的 protocol 為 Xmodem；其 update transport 為 UART。 資料腳位相同不代表 UART framed WRITE 可取代 Xmodem raw block。 設定條件見 通訊設定與更新必要條件。

Boot 執行流程



Boot 更新以 staged Metadata 與完整 BIN size 綁定串流，排除末塊 padding。 App / Transfer 的 STREAM_TEST 使用同一 block 接收方式，但資料只送入 RAM CRC32 sink。

Protocol Format

Byte / 控制	意義
0	SOH01
1 / 2	8-bit block number / ones complement，可wrap
3..130	128-byte payload，末塊unused positions填1A
131 / 132	CRC16/XMODEM high / low byte，poly1021、init0
C / ACK / NAK / CAN	43 / 06 / 15 / 18；兩個CAN取消
EOT04	完整映像或診斷CRC32 finalization，只送一次

階段	線上格式	判斷依據
準備	共用 ISP framed INFO / ENTER / METADATA / START	共用 header、命令與 INFO
Raw data	SOH、block number、complement、128 bytes、CRC16，共 133 bytes	CRC16 只覆蓋 128-byte payload，尾碼 high byte 先送
末塊	不足 128 bytes 的位置填 1A	只提交 START 宣告的 useful bytes
結束	EOT 與最終 ACK	Boot 完整 BIN CRC32 / Metadata / IRQ 驗證通過後才完成

CRC16/XMODEM 使用 polynomial 1021h、initial 0000h、無反射、無 final XOR；`123456789` 的結果為 31C3h。 它保護單一 block；共用控制 frame 與完整映像仍各自使用 CRC32。

Block 範例

```
01 01 FE [payload: 00 01 02 ... 7F   128 bytes] E8 0A
```

第一個 01 是 SOH，第二個 01 是 block number，FE 是其 ones complement。 E8 0A 為這組 128 bytes 的 CRC16 high / low byte；block number 以 8-bit wrap，從 FF 接到 00。

成功與錯誤回覆範例

Request / 狀態	Device回覆	意義
上述block通過CRC、順序與寫入	`06` ACK	可以送下一block；完整映像仍待EOT驗證
CRC錯誤或partial block超時	`15` NAK	重送相同block；不跳到下一block
完整相同duplicate	`06` ACK	重播接受結果，不再次寫入
EOT且完整映像驗證通過	`06`最終ACK，傳送完成後reset	Tool另行確認App ready
取消或完整映像驗證失敗	`18 18` CAN CAN	不以ACK或重啟推定成功；保存結果後查詢狀態

ACK / NAK是raw單byte控制，不含ISP frame CRC；framed錯誤回覆只在framed控制階段使用。

異常與完成條件

條件	處理	完成判斷
Block CRC16 / 順序錯誤	NAK；Tool 對相同 block 最多三次嘗試	尚未提交錯誤 block
精確 duplicate block	全部 128-byte 內容相同才重播 ACK	不重複寫入 Flash
Partial block 閒置 200 ms	NAK 並清除 partial block	等待合法 block
Streaming idle 10 秒或收到兩個 CAN	取消串流，返回 framed parser	不執行 pending reset
本機 C / ACK / NAK 傳送失敗	取消串流並清除 pending reset	不以未完成回覆授權重啟
EOT 完整驗證成功	最終 ACK 完整傳送後才 reset	Tool 另行確認重連與 App ready

EOT 只送一次；最終 ACK 遺失時保留映像與報告，再查詢即時狀態，不以重送 EOT 或重燒推定結果。 平台與實板驗證範圍見 驗證範圍。

程式對照與診斷

程式位置	入口或 API	責任
`custom_func.c`	`hardware_init()` / `loop()`、串流 callback	framed / raw 模式切換、sink 與回覆完成後動作
`hal/isp/isp_protocol.c` / `isp_device.c`	`IspStreamFeed()` / `IspDeviceProcess()`	Metadata、START、版本與角色契約
`hal/isp/isp_xmodem.c`	`IspXmodemInit()` / `IspXmodemFeed()` / `IspXmodemPoll()`	block、CRC16、duplicate、cancel 與接收期限
同一 Xmodem 模組	`IspXmodemCrc()`	CRC16/XMODEM
`hal/image` / `hal/flash`	映像驗證與 Flash API	Boot 寫入及完整 BIN 驗證；診斷 sink 不使用 Flash

Tool 的 Xmodem Transfer 使用 389 useful bytes，涵蓋末塊填充及 CRC32。 UART queue 與硬體錯誤欄位沿用 UART 診斷格式。 Tool 操作與報告集中於 串列操作。